

Roadmap complet: Baze de date, Elasticsearch și Data Platform

PostgreSQL · Redis · Search · NoSQL · CDC · Object Storage · OLAP · 40 de săptămâni

Scopul documentului

Acest curriculum este continuarea practică a roadmap-ului **Java + Spring Boot + Kafka**. El acoperă datele de care are nevoie un backend modern: baze relaționale, cache distribuit, motoare de căutare, baze NoSQL, object storage, CDC, time-series, OLAP și căutare vectorială.

Ținta nu este să aduni logo-uri în CV. La final trebuie să poți decide unde trăiește adevărul, ce sistem este doar o proiecție derivată, cum păstrezi consistența, cum măsoară performanța și cum recuperezi datele după un incident. Trebuie să poți trece de la cerința „căutarea este lentă” la model, index, pipeline, SLO, teste și runbook.

Documentul a fost cercetat în august 2026 și folosește în principal documentațiile oficiale ale tehnologiilor. Referințele [S1]-[S50] sunt la final.

Profilul profesional urmărit

Axă	Ce trebuie demonstrat	Pondere
SQL și model relațional	Scheme, constrângeri, interogări, tranzacții	25%
PostgreSQL în producție	Indexuri, planuri, MVCC, vacuum, backup	20%
Search și Elasticsearch	Mapping, relevanță, ingestie, operare	20%
Data platform	Redis, CDC, object storage, NoSQL, OLAP	20%
Engineering	Spring, migrații, securitate, teste, SRE	15%

Rezultatul urmărit

La final trebuie să poți demonstra că:

- alegi între PostgreSQL, Elasticsearch, Redis, MongoDB, Cassandra, object storage și un sistem analitic pe baza accesului la date;
- modelezi o schemă relațională cu invariants impuse în bază, nu doar în cod;
- citești **EXPLAIN ANALYZE** și corectezi un plan slab prin rescriere, indexare sau modelare;
- explici MVCC, isolation levels, locks, deadlocks, WAL, vacuum, replicare și PITR;
- proiectezi mapping-uri Elasticsearch explicite și un set măsurabil de teste de relevanță;
- sincronizezi PostgreSQL cu Elasticsearch fără dual writes fragile;
- folosești Redis ca accelerator, nu ca substitut accidental pentru o bază durabilă;
- proiectezi versionare, lifecycle, securitate și upload pentru obiecte mari;
- alegi un model document, wide-column, time-series sau columnar numai când accesul îl justifică;
- faci restore drills, teste de migrare, load tests și reconciliation;
- estimezi RPO, RTO, capacitate, cost și impactul unei defecțiuni;
- explici compromisurile stakeholderilor în termeni de latență, prospețime, risc și bani.

Cum folosești curriculumul

Durată și ritm

Planul de bază are **40 de săptămâni**, la 12-15 ore pe săptămână. Un backend developer experimentat poate comprima SQL-ul introductiv, dar nu ar trebui să sară peste tranzații, restore, relevanță, CDC sau operare.

O săptămână tipică:

- 3 ore de documentație și concepte;
- 7-9 ore de implementare și experimente;
- 2 ore de măsurare și defectare controlată;
- 1 oră de ADR, diagramă, runbook sau postmortem;
- 1 oră de recapitulare orală.

Regula de promovare

Un subiect este învățat doar când ai patru dovezi:

- 1 **Explicație:** îl explici simplu și îi descrii limitele.
- 2 **Implementare:** construiești un exemplu fără copy-paste orb.
- 3 **Măsurare:** definești metrice, un baseline și un prag.
- 4 **Diagnostic:** reproduci o defecțiune și o izolezi.

Diagnostic inițial

Înainte să începi, încearcă fără tutorial:

- modelează comenzi, produse și stoc, cu chei și constrângeri;
- scrie o interogare cu join, agregare și funcție window;
- explică de ce un index compus (**tenant_id, status, created_at**) nu servește orice query;
- interpretează un plan cu scanare secvențială, nested loop și sort;
- previne două rezervări concurente ale ultimei unități;
- recuperează un PostgreSQL de test la un moment anterior;
- modelează același produs în Elasticsearch pentru text și filtrare exactă;
- implementează cache-aside și demonstrează cum apare un stampede;
- explică de ce replicarea nu este backup;
- sincronizează o modificare SQL către un read model fără să pierzi evenimentul.

Punctele ratate devin obligatorii. Pentru cele reușite, fă direct gate-ul practic.

Capitolul 1 - Modelul mental al unei platforme de date

1.1 Pornește de la acces, garanții și cost

Pentru orice alegere, scrie întâi:

- forma datelor și relațiile dintre ele;
- operațiile dominante: lookup, range, full-text, agregare, append;
- volum actual, rată de creștere și distribuția cheilor;
- latența și throughput-ul cerut la percentile, nu doar media;
- consistența acceptată și cât de vechi pot fi rezultatele;
- durabilitatea, RPO și RTO;
- modelul de securitate și rezidența datelor;
- competența echipei și costul de operare.

Nu întreba „care bază este cea mai rapidă?”. Întreabă „rapidă pentru ce query, la ce scară, cu ce garanții și ce cost operațional?”.

1.2 Rolurile tehnologiilor

Necesitate dominantă	Alegere de pornire	Motiv
Tranzacții și relații	PostgreSQL/MySQL	Constrângeri, SQL, ACID
Full-text și relevanță	Elasticsearch	Index inversat, analiză, ranking
Cache și stare efemeră	Redis	Acces rapid și TTL
Fișiere și blob-uri	S3/Object Storage	Durabilitate și scalare pentru obiecte
Documente autonome	MongoDB	Agregate document și schemă flexibilă
Scrieri masive distribuite	Cassandra	Partiționare și disponibilitate
Serii temporale	TimescaleDB	Partiționare temporală și agregări
Analytics columnar	ClickHouse	Scanări și agregări OLAP

Tabelul este un punct de pornire, nu o rețetă. PostgreSQL poate acoperi JSON, full-text, vectori și time-series la o scară considerabilă. Introdu o tehnologie nouă numai când beneficiul măsurat depășește costul de operare.

1.3 Source of truth și proiecții

Definește explicit:

- **system of record:** autoritatea care decide starea validă;
- **read model:** proiecție optimizată pentru un tip de citire;
- **cache:** copie temporară care poate fi reconstruită;
- **event log:** istoric ordonat în limitele cheii/partiției;
- **data lake/warehouse:** copie pentru analiză, nu tranzacții operaționale.

Un model sănătos pentru catalog este: PostgreSQL decide produsul și prețul, Elasticsearch servește căutarea, Redis accelerează paginile fierbinți, S3 ține imaginile. Dacă Elasticsearch dispare, îl reconstruiești; nu inventezi adevărul din index.

Gate 1

Scrie un ADR pentru trei sisteme: magazin online, documente juridice și telemetrie IoT. Include workload, garanții, diagramă a fluxului, source of truth, failure modes și costul unei tehnologii în plus.

Capitolul 2 - Modelare relațională și SQL

2.1 Modelul relațional

Stăpânește:

- tabele, tuples, attribute, domains și chei candidate;
- primary key, foreign key, unique, check și not null;
- relații 1:1, 1:N, N:M și tabele de asociere;

- entitate, value object, agregat și istoric;
- identificatori naturali versus surrogate keys;
- modelarea banilor, fusurilor orare și statusurilor;
- soft delete, temporalitate și audit;
- null și logica ternară.

Invarianta critică trebuie impusă cât mai aproape de date. Un **UNIQUE(tenant_id, external_id)** apără sistemul de toate căile de scriere; un **if** într-un singur serviciu nu.

2.2 Normalizare și denormalizare

Învăță 1NF, 2NF, 3NF și BCNF pentru a recunoaște dependențe și anomalii. Normalizează implicit pentru OLTP, apoi denormalizează selectiv când ai:

- un query critic măsurat;
- un ownership clar al câmpului duplicat;
- mecanism de actualizare și reconciliation;
- strategie de backfill;
- toleranță explicită la staleness.

Duplicarea necontrolată produce surse concurente de adevăr. Denormalizarea controlată produce un read model.

2.3 SQL de bază și avansat

Trebuie să scrii fluent:

- **SELECT, INSERT, UPDATE, DELETE, RETURNING** și upsert;
- inner/outer joins, semi-joins și anti-joins;
- **GROUP BY, HAVING** și agregări condiționale;
- subqueries corelate și necorelate;
- CTE-uri și CTE-uri recursive;
- window functions: **row_number, rank, lag, lead**, running totals;
- set operations: union, intersect, except;
- keyset pagination;
- JSON și array operators atunci când modelul le justifică.

Înțelege ordinea logică a unei interogări și diferența dintre rezultat corect și plan eficient.

2.4 Tipuri și precizie

- Folosește **numeric** sau minor units pentru bani; nu **float**.
- Stochează momentele ca **timestampz/Instant**; păstrează separat zona dacă este cerință de business.

- Folosește UUID când distribuția/integrarea îl cere, dar înțelege impactul cheilor aleatorii asupra indexurilor.
- Pune dimensiune și semantică pe stringuri doar când reprezintă o regulă reală.
- Evită coloane generice **data**, EAV și JSON pentru proprietăți centrale interogate frecvent.

Exerciții

Modelează un marketplace multi-tenant. Adaugă comenzi, linii, plăți, stoc și audit de preț. Scrie 20 de query-uri, inclusiv top produse pe lună, cohoți de clienți, stoc disponibil și keyset pagination. Generează date suficiente încât planurile să fie relevante.

Gate 2

Livrează schema prin migrații, un dicționar de date, invariants testate și un raport cu cinci query-uri avansate. Pentru fiecare query: rezultat așteptat, plan, cardinalități și timp la un volum declarat.

Capitolul 3 - PostgreSQL: arhitectură internă

3.1 Procese, memorie și pagini

Înțelege conceptual:

- client, backend process, shared buffers și OS page cache;
- pagini, heap tuples și index pages;
- buffer cache hit versus acces la disc;
- checkpoints, background writer și costul scrierilor;
- catalogul de sistem și statisticile plannerului.

Nu optimizez parametri la întâmplare. Stabilește întâi query-ul, volumul, I/O, CPU, working set și concurența.

3.2 MVCC

PostgreSQL folosește Multi-Version Concurrency Control: fiecare statement vede un snapshot, iar versiunile de rând permit citirilor și scrierilor să progreseze fără blocarea grosieră tipică [S1]. Învață:

- tuple versions și vizibilitate;
- **xmin/xmax** la nivel conceptual;
- snapshots și efectul nivelului de izolare;
- dead tuples și bloat;
- de ce un update creează de regulă o nouă versiune;
- HOT updates și impactul indexurilor;
- long-running transactions care împiedică cleanup-ul.

3.3 WAL și checkpoints

Write-Ahead Log este jurnalul schimbărilor care trebuie persistat înaintea paginilor de date. El stă la baza crash recovery, replicării și point-in-time recovery [S7]. Învăță:

- WAL records, LSN și segmente;
- checkpoint și recovery;
- **synchronous_commit** ca trade-off;
- archive mode și retenția WAL;
- replica lag și slots care pot reține WAL;
- de ce un volum mare de update poate deveni problemă de I/O.

3.4 Vacuum, autovacuum și statistics

VACUUM recuperează spațiul reutilizabil, actualizează visibility map și previne wraparound; **ANALYZE** alimentează plannerul cu statistici [S2]. Trebuie să știi:

- vacuum standard versus **VACUUM FULL**;
- autovacuum thresholds și scale factors;
- freeze și transaction ID wraparound;
- bloat, dead tuple ratio și tables hot spots;
- statistics target și coloane corelate;
- de ce oprirea autovacuum-ului nu este o optimizare.

Gate 3

Pornește o bază de test, execută update-uri masive și o tranzacție lungă. Urmărește dead tuples, mărirea tabeli, autovacuum și WAL. Scrie diagnosticul și măsurile de remediere fără să folosești **VACUUM FULL** ca reflex.

Capitolul 4 - Indexuri și optimizarea interogărilor

4.1 Cum gândește plannerul

Plannerul estimează cardinalități și costuri, apoi alege scanări, join-uri, sortări și agregări.

EXPLAIN ANALYZE execută interogarea și arată estimări versus valori reale [S3]. Citește:

- estimated/actual rows și loops;
- startup/total cost și actual time;
- sequential, index și bitmap scans;
- nested loop, hash join și merge join;
- sort methods și spill pe disc;
- buffers, I/O și planning/execution time;

- filter rows removed;
- paralelism.

Folosește **EXPLAIN (ANALYZE, BUFFERS)** pe un mediu sigur. Pentru **UPDATE/DELETE**, rulează într-o tranzacție pe care o poți rollback-ui.

4.2 Familii de indexuri

PostgreSQL oferă B-tree, Hash, GiST, SP-GiST, GIN și BRIN, fiecare pentru clase diferite de operatori [S4]. Învăță:

- B-tree pentru egalitate, range și ordering;
- indexuri compuse și regula prefixului stâng;
- selectivitate și corelație;
- covering indexes cu **INCLUDE**;
- partial și expression indexes;
- GIN pentru array, JSONB și full-text;
- GiST/SP-GiST pentru geospațial și structuri specializate;
- BRIN pentru tabele foarte mari corelate cu ordinea fizică;
- costul fiecărui index la write, vacuum și storage.

4.3 Pattern-uri care strică planurile

- **SELECT *** și over-fetching;
- funcții/casturi pe coloana indexată;
- wildcard cu prefix necunoscut;
- paginare cu offset mare;
- N+1 din ORM;
- condiții OR și joins care multiplică rândurile;
- statistică veche sau date puternic skewed;
- tranzacții care țin locks și conexiuni prea mult;
- indexuri redundante ori aproape identice.

Nu forța un index înainte să înțelegi estimatorul. O scanare secvențială poate fi corectă dacă query-ul citește mare parte din tabel.

Gate 4

Ia zece query-uri lente pe minimum un milion de rânduri. Păstrează baseline, planurile și distribuțiile. Îmbunătățește-le fără să adaugi mai mult de cinci indexuri și arată costul scrierilor înainte/după.

Capitolul 5 - Tranzacții, concurență și corectitudine

5.1 ACID și isolation levels

Înțelege atomicitate, consistență, izolare și durabilitate, apoi separă consistența bazei de consistența de business. Studiază:

- Read Committed, Repeatable Read și Serializable;
- dirty read, non-repeatable read, phantom și serialization anomaly;
- snapshot isolation și write skew;
- retry pentru serialization failures;
- autocommit și limite tranzacționale.

5.2 Locks și strategii de concurență

- row/table locks și lock modes;
- **SELECT ... FOR UPDATE** și **SKIP LOCKED**;
- optimistic locking cu version column;
- advisory locks pentru coordonare atent delimitată;
- deadlock detection și ordinea consistentă a locks;
- tranzacții scurte și fără I/O extern în interior.

Pentru rezervarea stocului, compară update atomic condiționat, pessimistic lock și optimistic retry. Alege pe baza conflict rate și latenței, nu a preferinței personale.

5.3 Idempotency și exact-once la granițe

O operație idempotentă produce aceeași stare la repetare. Folosește:

- idempotency key cu rezultat memorat;
- constrângeri unique;
- compare-and-set/version;
- inbox pentru mesaje procesate;
- outbox în aceeași tranzacție cu schimbarea de business;
- stări monotone și commutative unde se poate.

„Exactly once” end-to-end trebuie descompus: producer, broker, consumer, baza țintă și side effects. De multe ori soluția reală este at-least-once plus idempotency.

Gate 5

Construiește un serviciu de rezervări. Rulează 100 de requesturi concurente pentru 10 unități, injectează timeout după commit și demonstrează că nu există oversell sau duplicate. Compară două strategii și documentează throughput-ul.

Capitolul 6 - Acces la date din Java și Spring

6.1 JDBC, pooling și limite

Învățã traseul request - connection pool - server - storage. Configurează și măsoară:

- pool size raportat la capacitatea bazei;
- acquisition timeout, statement timeout și socket timeout;
- connection lifetime și validation;
- prepared statements și batching;
- tranzacții și cleanup;
- query tags/application name pentru diagnostic.

Mai multe conexiuni nu înseamnă automat throughput mai mare. Pot crește context switching, lock contention și coada din bază.

6.2 JPA/Hibernate fără surprize

Stăpânește:

- entity lifecycle, persistence context și dirty checking;
- mapping pentru value objects și relații;
- fetch plan, lazy/eager și N+1;
- cascades și orphan removal;
- batching și bulk operations;
- optimistic locking cu **@Version**;
- projections și DTO-uri;
- limitele pagination + collection fetch.

Activează observabilitatea SQL în development/test. Nu expune entități direct prin API.

6.3 Alternative explicite

Spring **JdbcClient**, jOOQ sau SQL nativ pot fi mai clare pentru raportare, query-uri complexe și control fin. Alege per use case; un proiect poate folosi JPA pentru agregate și SQL explicit pentru read models.

6.4 Testare

- unit tests pentru logică pură;
- integration tests cu PostgreSQL real prin Testcontainers;
- teste de repository pentru mappings, constraints și locking;
- dataset mic lizibil plus generator de volum;
- teste de timeout și epuizare a pool-ului;
- verificarea query count pentru N+1.

Gate 6

Expune un API Spring pentru marketplace. Demonstrează un N+1, repară-l în două moduri, măsoară query count și latența. Adaugă locking, idempotency și teste concurente cu baza reală.

Capitolul 7 - Migrații și evoluția schemei

7.1 Migrații versionate

Folosește Flyway sau Liquibase și învață:

- migrații forward-only și checksums;
- ownership și review;
- DDL tranzacțional versus operații cu lock lung;
- preconditions și validare;
- separarea schimbării de schemă de deploy-ul aplicației;
- testarea pornind de la versiunea de producție.

Nu modifica o migrație deja aplicată. Creează una nouă și păstrează istoricul reproductibil.

7.2 Expand-contract

Pentru schimbări compatibile:

- 1 adaugă noua structură fără să o ceri imediat;
- 2 deployează cod care poate citi vechi și nou;
- 3 backfill în batch-uri cu checkpoint;
- 4 validează și reconciliază;
- 5 mută citirea pe nou;
- 6 oprește scrierea veche;
- 7 elimină vechiul câmp într-un deploy ulterior.

Evită dual writes fără mecanism de recuperare. Pentru tabele mari, înțelege lock-urile DDL și operațiile online disponibile versiunii tale.

7.3 Backfill sigur

Un backfill trebuie să fie reluiabil, throttled, observabil și să nu monopolizeze locks/WAL.

Include:

- intervale sau keyset cursor;
- batch size configurabil;
- rate limit și pauză la presiune;
- idempotency;
- progres persistent;
- verificare checksum/count;

- plan de rollback sau roll-forward.

Gate 7

Schimbă într-o aplicație activă un câmp **full_name** în **first_name/last_name**, cu două versiuni simultane, backfill și rollback. Apoi adaugă un index mare fără downtime observabil și documentează lock-urile.

Capitolul 8 - Replicare, backup și disaster recovery

8.1 Disponibilitate și replicare

Învăță:

- primary/replica și streaming replication;
- synchronous versus asynchronous replication;
- replica lag și read-after-write;
- failover, split brain și fencing;
- connection routing și DNS/service discovery;
- replication slots și retenția WAL;
- read replicas pentru scalare selectivă.

O replică reproduce și ștergerea accidentală. De aceea replicarea nu este backup.

8.2 Backup și PITR

PostgreSQL combină base backup și secvența WAL pentru point-in-time recovery [S7].
Definește:

- RPO: câtă informație poți pierde;
- RTO: cât poate dura recuperarea;
- full/incremental/logical backup;
- criptare, retenție, locație separată și acces;
- catalog și monitorizarea backup jobs;
- restore până la timestamp/LSN;
- verificare automată și restore drills.

Un backup netestat este o ipoteză. Măsoară timpul de restore cu volumul realist și include reconectarea aplicației.

Gate 8

Șterge intenționat date într-un mediu izolat. Recuperează la momentul anterior, validează integritatea și scrie un runbook cu RPO/RTO măsurate, responsabilități și criterii de oprire.

Capitolul 9 - Redis și caching distribuit

9.1 Ce este Redis în arhitectură

Redis oferă structuri în memorie: strings, hashes, sets, sorted sets, streams, probabilistic structures și altele [S14]. Folosește-l pentru:

- cache;
- rate limiting;
- session/token state atent proiectată;
- counters și leaderboards;
- locks numai cu înțelegerea garanțiilor;
- cozi/streams când cerințele se potrivesc.

Nu-l transforma în source of truth doar fiindcă răspunde repede.

9.2 Cache-aside

Fluxul uzual este: citește cache, la miss citește DB, populează cache; la write actualizează DB și invalidează cache [S16]. Învață:

- key design și namespacing;
- serialization/versioning;
- TTL cu jitter;
- negative caching;
- invalidation și staleness budget;
- cache stampede și request coalescing;
- hot keys și distribuție;
- cache warming;
- măsurarea hit ratio împreună cu latența și costul.

Un hit ratio mare poate ascunde obiecte enorme sau date vechi. Optimizează impactul, nu procentul izolat.

9.3 Memorie, eviction și durabilitate

Configurează **maxmemory** și o politică de eviction potrivită workload-ului [S15]. Înțelege:

- LRU/LFU aproximativ și politici volatile/allkeys;
- TTL expiration versus eviction sub presiune;
- fragmentare și overhead per key;
- RDB snapshots versus AOF;
- replicație asincronă;
- Sentinel pentru monitorizare/failover și limitele sale [S12];
- Redis Cluster, hash slots și multi-key constraints.

Chiar cu Sentinel, scrieri confirmate se pot pierde în anumite ferestre de failover; proiectează după garanția reală, nu după eticheta „HA”.

9.4 Spring Data Redis

Folosește **RedisTemplate**, repositories sau Spring Cache conștient de serializare, TTL și tranzacționalitate [S18]. Evită Java native serialization. Include timeout, circuit breaker și fallback: cache-ul indisponibil nu trebuie să doboare automat source of truth.

Gate 9

Adaugă Redis în proiectul marketplace. Simulează miss storm, hot key, eviction și Redis indisponibil. Demonstrează TTL jitter, single-flight și degradare controlată. Compară p50/p95/p99 și load-ul pe PostgreSQL.

Capitolul 10 - Elasticsearch: fundația

10.1 Modelul de căutare

Elasticsearch este un motor distribuit construit peste Lucene. Modelul său este document-oriented și optimizat pentru căutare. Învăță:

- index, document, field și mapping;
- index inversat, terms și postings;
- segments și merge;
- near-real-time refresh;
- primary shards și replicas;
- routing și distribuția documentelor;
- coordinator, data și master-eligible nodes la nivel conceptual;
- cluster state.

Elasticsearch nu este un înlocuitor implicit pentru baza tranzacțională. Update-ul unui document este intern o nouă versiune, relațiile și tranzacțiile multi-document sunt limitate, iar mapping-ul și shard-urile cer design anticipat.

10.2 Shards și replicas

Un shard este o unitate Lucene, nu doar o setare de paralelism. Prea multe shard-uri adaugă overhead; shard-uri prea mari încetinesc recovery și rebalancing. Elastic recomandă benchmark cu datele, hardware-ul și query-urile reale, nu un număr magic [S30].

Decide:

- volum și rată de ingestie;
- retenție și rollover;
- query fan-out;
- numărul/natura nodurilor;
- recovery time;

- necesitatea replicas pentru availability și read capacity.

10.3 Document modeling

Denormalizează pentru query. Înțelege:

- object versus **nested**;
- arrays și pierderea asocierii între câmpuri la objects;
- parent-child doar pentru cazuri justificate;
- câmpuri duplicat controlat;
- version field și source identifiers;
- document size și update frequency.

Gate 10

Indexează un milion de produse sintetice. Variaza shard count, replicas și refresh interval într-un mediu izolat. Măsoară bulk throughput, query latency, storage și recovery; formulează o alegere justificată.

Capitolul 11 - Mapping, analiză și schema Elasticsearch

11.1 text versus keyword

Un câmp **text** este analizat pentru full-text; **keyword** păstrează valoarea pentru filtrare exactă, sortare și agregare [S22]. Multi-fields permit aceeași valoare în forme diferite [S23]. Exemple:

- **name: text** cu subcâmp **keyword**;
- **sku, tenantId, status: keyword**;
- **price:** numeric;
- **createdAt:** date;
- descriere: **text**, fără agregări;
- obiecte corelate în array: **nested**.

11.2 Analyzer pipeline

Analyzer-ul combină character filters, tokenizer și token filters [S20]. Învăță:

- standard, language analyzers și custom analyzers;
- lowercase, asciifolding și stemmers;
- stop words și riscul pierderii de sens;
- edge n-gram pentru autocomplete;
- synonyms la index time versus search time;
- normalizers pentru keyword;
- **search_analyzer** diferit de index analyzer;

- **_analyze** pentru testare.

Folosește limba și vocabularul domeniului. „Căutare română” cere teste cu diacritice, forme flexionare, nume, coduri și greșeli reale.

11.3 Mapping explicit și evoluție

Dynamic mapping poate detecta câmpuri noi [S21], dar în producție poate crea type conflicts, mapping explosion și cost. Preferă:

- mappings explicite;
- dynamic **strict** sau template-uri controlate;
- index templates și component templates;
- aliases pentru schimbări;
- index versionat **products-v003**;
- reindex și switch atomic de alias;
- compatibilitate între producători și mapping.

Tipul unui câmp existent nu se schimbă magic. Creezi un index nou, reindexezi, validezi și muți aliasul.

Gate 11

Construiește un index bilingv de produse cu exact matching, full-text, autocomplete, filtre și nested variants. Scribe minimum 30 de teste de analiză și mapping, inclusiv diacritice, SKU, sinonime și valori necunoscute.

Capitolul 12 - Query DSL, relevanță și evaluare

12.1 Queries și filters

Învăță:

- **match**, **multi_match**, **match_phrase** și full-text queries [S25];
- **term**, **terms**, range și exists pentru date structurate;
- **bool** cu **must**, **should**, **filter** și **must_not** [S24];
- query context versus filter context;
- minimum_should_match;
- boosting și function score;
- aggregations și post-filter;
- highlighting;
- search after și point-in-time pentru paginare adâncă.

Nu folosi **term** pentru text analizat și nu folosi **match** ca substitut accidental pentru filtrarea unui ID.

12.2 BM25 și relevanță

BM25 este similaritatea implicită pentru câmpuri text [S26]. Înțelege intuitiv:

- term frequency;
- inverse document frequency;
- document length normalization;
- field boosts;
- efectul analyzer-ului asupra scoring-ului.

Relevanța nu se optimizează „pare bine”. Creează un golden query set din trafic și feedback:

- query și context;
- documente relevante cu grade;
- cazuri zero-result;
- segmente/limbi;
- reguli de business separate de relevanța lexicală.

Măsoară precision@k, recall@k, MRR, NDCG și zero-result rate. Urmărește și p95/p99, deoarece o îmbunătățire de ranking inutil de lentă nu este produs bun.

12.3 Sinonime, fuzziness și autocomplete

- Sinonimele sunt reguli de produs și au nevoie de versionare/evaluare.
- Fuzziness ajută la typo, dar poate crește mult candidații și rezultate greșite.
- Prefix și wildcard sunt costisitoare în formele nepotrivite.
- Autocomplete poate folosi edge n-grams, completion suggerer sau search-as-you-type; benchmark pe vocabular real.
- Popularitatea și recency pot fi semnale, dar trebuie prevenit feedback loop-ul.

Gate 12

Construiește 100 de interogări etichetate pentru catalog. Compară trei configurații. Publică metricile, query latency și exemplele de regresie. Un candidat trebuie să bată baseline-ul fără să înrăutățească segmente critice.

Capitolul 13 - Ingestie și sincronizare DB - Elasticsearch

13.1 Bulk și refresh

Folosește Bulk API cu batch-uri măsurate, backpressure și retry numai pentru erori retryable. Înțelege:

- document IDs deterministe;
- create/index/update/delete semantics;

- partial failures într-un răspuns bulk;
- refresh interval și vizibilitate;
- versioning/external sequence;
- dead-letter și replay;
- throttling pentru a proteja clusterul.

13.2 De ce dual write este fragil

Fluxul „scrie PostgreSQL, apoi Elasticsearch” are ferestre inevitabile:

- DB commit reușește, indexarea eșuează;
- indexarea reușește, DB transaction dă rollback;
- timeout ascunde rezultatul;
- retry schimbă ordinea;
- delete-ul nu ajunge;
- deploy-urile rulează scheme diferite.

Soluția uzuală este transactional outbox: schimbarea de business și rândul outbox sunt în aceeași tranzacție. Debezium citește logul bazei și publică evenimentul; Outbox Event Router folosește ID pentru deduplicare și aggregate ID drept cheie pentru ordering [S37].

13.3 Pipeline robust

Include:

- event ID, aggregate ID/type și schema version;
- poziție/versiune monotonă per agregat;
- consumer idempotent;
- upsert/delete/tombstone;
- retry cu backoff și DLT;
- monitoring pentru lag și failure rate;
- periodic reconciliation DB - index;
- full rebuild într-un index nou;
- alias switch după validare;
- replay fără efecte duble.

Pentru o reconstrucție, capturează un punct consistent, indexează snapshot-ul, aplică delta de la acel punct, apoi comută aliasul.

13.4 Spring Data Elasticsearch

Repository abstractions sunt utile pentru operații simple, iar **ElasticsearchOperations** oferă control mai explicit [S31][S32]. Păstrează modelul indexului separat de entity JPA. Query-urile complexe și bulk indexing merită cod explicit și teste de integrare cu versiunea reală a clusterului.

Gate 13

Sincronizează catalogul prin outbox + Debezium. Oprește Elasticsearch, consumerul și rețeaua pe rând. Repornește, reconciliază și demonstrează că indexul converge fără documente pierdute sau versiuni vechi.

Capitolul 14 - Elasticsearch în producție

14.1 Capacity și sănătatea clusterului

Urmărește:

- heap și GC;
- disk usage și watermarks;
- CPU, load și I/O;
- indexing/search latency și rejection;
- segment count, merge time și refresh;
- shard allocation și unassigned shards;
- cluster state size;
- query cache/request cache;
- recovery și relocation.

Testează cu date, concurență și distribuție realistă. p99, merge-urile și recovery sunt adesea mai importante decât un benchmark single-query.

14.2 ILM, data streams și retenție

Index Lifecycle Management automatizează rollover, shrink/force merge, mutare pe tier-uri și delete în faze [S27][S28]. Pentru date time-series, înțelege data streams și backing indexes. Politica trebuie să reflecte:

- retenția legală și de produs;
- rata de ingestie;
- dimensiunea/vechimea pentru rollover;
- tier-uri hot/warm/cold/frozen;
- capacitatea de rehidratare;
- cost.

14.3 Snapshots și restore

Snapshots sunt mecanismul de backup al clusterului [S29]. Configurează repository în storage extern, retenție, monitorizare și restore drill. Nu copia manual directorul de date. Verifică ce include snapshot-ul, compatibilitatea versiunilor și timpul de restore.

14.4 Security și upgrades

- TLS între clienți/noduri;

- autentificare și roluri cu least privilege;
- field/document level security dacă licența și cazul o cer;
- secrets în secret manager;
- audit logging;
- protejarea endpoint-urilor de query scumpe;
- rolling upgrade conform matricei de compatibilitate;
- testarea clientului Spring cu versiunea țintă;
- snapshot și rollback/roll-forward plan.

Gate 14

Scrie un runbook pentru disk watermark, unassigned shard, heap pressure și query lent. Execută un restore într-un cluster separat și măsoară RTO. Simulează pierderea unui nod și urmărește relocarea.

Capitolul 15 - MongoDB și modelarea documentelor

15.1 Când se potrivește

MongoDB este util când datele formează agregate document citite împreună, structura evoluează și relațiile cross-aggregate sunt limitate. Embedding poate returna datele asociate într-o singură operație și update-ul unui document este atomic [S33].

Învață:

- embed versus reference;
- document de maximum 16 MiB;
- schema validation;
- compound/multikey indexes;
- replica sets, read/write concerns;
- transactions multi-document și costul lor [S34];
- sharding și alegerea shard key [S35].

15.2 Capcane

- documente care cresc fără limită;
- duplicare fără owner;
- shard key monotonă care creează hotspot;
- query-uri care nu includ cheia de distribuție;
- indexuri multikey neașteptate;
- folosirea tranzacțiilor pentru a imita un model relațional complex.

Gate 15

Modelează un catalog cu variante și attribute flexibile în PostgreSQL JSONB și MongoDB. Compară query-uri, invariants, update contention, migrare și operare. Scrie criteriul obiectiv de alegere.

Capitolul 16 - Cassandra și wide-column design

Cassandra este peer-to-peer, distribuită și orientată către availability și scalare orizontală [S39]. Modelarea este query-first:

- partition key decide localitatea datelor;
- clustering columns decid ordinea în partiție [S40];
- denormalizezi în tabele per query;
- consistența este configurabilă per operație;
- scrierile trec prin commit log/memtable și ajung în SSTables;
- compaction gestionează SSTables și tombstones [S41];
- repairs și anti-entropy sunt operații esențiale.

Învață:

- replication factor și consistency levels;
- quorum arithmetic;
- partition sizing și hot partitions;
- LSM tree, bloom filters și compaction strategies;
- tombstones, TTL și gc grace;
- lightweight transactions și costul Paxos;
- noduri, tokens, rack/DC awareness și repair.

Nu alege Cassandra pentru „foarte multe date” generic. Alege-o pentru query-uri previzibile la scală distribuită, când accepți denormalizare și echipa poate opera repair/compaction.

Gate 16

Proiectează telemetrie pe device și interval temporal. Calculează mărimea partiției, testează o partiție fierbinte, TTL/tombstones și o strategie de compaction. Compară cu TimescaleDB.

Capitolul 17 - Object storage și documente mari

17.1 Modelul corect

Object storage păstrează bytes la o cheie; nu este filesystem POSIX și nici bază de query. Un model uzual:

- blob în S3-compatible storage;
- metadata, ownership, status și ACL în PostgreSQL;
- text extras și câmpuri căutabile în Elasticsearch;
- evenimente de procesare în Kafka/queue.

17.2 Upload, integritate și lifecycle

Învăță:

- pre-signed URLs cu permisiune și expirare limitată [S42];
- multipart upload pentru obiecte mari [S43];
- checksums, content length și content type validate;
- chei opace, nu nume furnizate direct de utilizator;
- versioning și lifecycle;
- replication și regiuni [S44];
- Object Lock/WORM pentru cerințe de retenție [S45];
- orphan cleanup și reconciliere DB - bucket.

Un upload este un workflow: **PENDING**, upload, verificare, scanare malware, **AVAILABLE** sau **REJECTED**. Nu marca obiectul disponibil înainte de validare.

17.3 Security

Aplică block public access, least privilege, encryption, key rotation, audit, VPC endpoints unde este cazul și validarea conținutului [S46]. Pre-signed URL este bearer token temporar: cine îl are îl poate folosi în limitele lui.

Gate 17

Construiește upload direct către storage, metadata tranzacțională, checksum, scanare asincronă și download autorizat. Testează upload abandonat, retry multipart, fișier malițios, ștergere și cleanup.

Capitolul 18 - CDC, Debezium și integrarea datelor

Change Data Capture citește schimbările din WAL/binlog și le transformă într-un flux. Învăță:

- snapshot inițial și tranziția la streaming;
- offsets/LSN și restart;
- before/after, operation type și tombstones;
- schema history și DDL evolution;
- transaction metadata și ordering;
- replication slots și disk risk;

- connector lag, backpressure și retries;
- SMT-uri și Outbox Event Router [S36][S37].

CDC versus outbox

- Raw CDC expune forma tabelelor și este bun pentru data integration controlată.
- Outbox publică un contract de domeniu intenționat și decuplează consumatorii de schema internă.
- Polling outbox este mai simplu în unele echipe, dar are latență și concurență de gestionat.
- CDC nu elimină nevoia de idempotency, compatibilitate și reconciliation.

Schema evolution

Folosește contracte versionate și schimbări backward-compatible. Adaugă câmpuri opționale înainte să le ceri, păstrează semantică stabilă și testează consumatorii. Pentru schimbări incompatibile, publică o versiune nouă și migrează explicit.

Gate 18

Rulează Debezium pentru PostgreSQL, cu snapshot și streaming. Generează schema changes, restarturi și lag. Monitorizează slotul, replay-ul și compatibilitatea. Demonstrează un full rebuild al read model-ului.

Capitolul 19 - Time-series și analytics columnar

19.1 OLTP versus OLAP

OLTP favorizează rânduri, update-uri punctuale și tranzacții. OLAP favorizează scanarea unor coloane, compresie și agregări pe multe rânduri. Nu pune dashboard-uri grele pe primary-ul tranzacțional fără buget și izolare.

19.2 TimescaleDB

Hypertables împart automat seriile în chunks după timp și, opțional, altă dimensiune [S47]. Continuous aggregates precomputează incremental agregări [S48]. Învață:

- time partitioning și chunk interval;
- retention și compression;
- continuous aggregates și refresh policy;
- late-arriving data;
- indexuri și query-uri pe interval;
- cardinalitate și dimensionalitate.

19.3 ClickHouse

ClickHouse este columnar și potrivit pentru analytics cu volum mare. Studiază:

- MergeTree și background merges;

- **ORDER BY** ca ordine fizică/sparse primary index;
- **PARTITION BY** în principal pentru lifecycle și data management;
- partition pruning și data skipping indexes;
- materialized views;
- batch inserts și costul inserțiilor mici;
- deduplication/eventual mutations;
- distributed tables și replication.

Ordinea coloanelor trebuie aleasă după filtrele comune și cardinalitate. Benchmark pe query mix, nu doar pe ingestie.

Gate 19

Trimite evenimente de comenzi într-un store analitic. Construiește dashboard pe 12 luni și compară PostgreSQL, TimescaleDB și ClickHouse pentru două workload-uri. Include storage, ingestie, p95 și complexitate operațională.

Capitolul 20 - Căutare vectorială și hybrid search

20.1 Embeddings și nearest neighbors

Un embedding este un vector numeric. Similaritatea se măsoară prin cosine, inner product sau distanță Euclidean, în funcție de model. Învăță:

- dimensionalitate și normalizare;
- exact search versus approximate nearest neighbor;
- HNSW și IVFFlat la nivel conceptual;
- recall-latency-memory trade-off;
- pre-filter versus post-filter;
- versionarea modelului de embeddings;
- re-embedding și index rebuild;
- evaluare cu set etichetat.

pgvector adaugă vector similarity în PostgreSQL și oferă exact și approximate indexes, inclusiv HNSW și IVFFlat [S38]. Elasticsearch oferă **dense_vector**, kNN și filtrare [S49].

20.2 Hybrid search

Lexical search găsește termeni exacti, coduri și nume; vector search ajută semantic. Hybrid combină ambele, de exemplu prin Reciprocal Rank Fusion, recomandată în ghidul Elastic [S50]. Evaluează:

- lexical baseline;
- vector baseline;

- fusion și ponderi;
- reranking;
- costul producerii embedding-ului;
- fallback când serviciul de embeddings nu răspunde.

Nu introduce vector search fără un set de relevance judgments. Demo-ul impresionant pe cinci query-uri nu demonstrează valoare.

Gate 20

Adaugă semantic search catalogului în pgvector și Elasticsearch. Compară exact/ANN, recall@10, NDCG, p95 și memorie. Construiește hybrid retrieval și explică alegerea finală.

Capitolul 21 - Securitate, privacy și guvernanta

21.1 Principii

- clasifică datele: public, intern, confidențial, sensibil;
- colectează minimul necesar;
- definește owner, scop, retenție și consumatori;
- autentifică servicii și aplică least privilege;
- criptează în tranzit și at rest;
- rotește credențiale și chei;
- separă tenants la fiecare nivel;
- loghează accesul administrativ;
- maschează datele în non-production.

21.2 Multi-tenancy

Compară:

- database per tenant;
- schema per tenant;
- shared schema cu **tenant_id**;
- row-level security;
- index Elasticsearch per tenant versus shared index cu filtering;
- keys/prefixes Redis și risc de leakage;
- bucket/prefix policies în object storage.

Orice query trebuie să poarte contextul tenantului. Testează explicit cross-tenant leakage. Nu te baza doar pe filtrul UI.

21.3 Ștergere și retenție

„Right to delete” înseamnă propagare către source of truth, read models, cache, index, object storage, analytics și eventual backup policy. Definește:

- workflow și status;
- tombstone/event;
- SLA de propagare;
- excepții legale;
- dovadă/audit fără a reține conținutul șters;
- reconcilieri.

Gate 21

Fă threat model pentru proiectul document search. Demonstrează autorizare pe PostgreSQL, Elasticsearch și S3, plus un workflow de ștergere end-to-end și teste de izolare între tenants.

Capitolul 22 - Observabilitate, capacitate și cost

22.1 SLI și SLO

Definește SLI pentru experiența utilizatorului și pentru subsisteme:

- query success și p50/p95/p99;
- freshness/replication/CDC lag;
- cache hit, miss, eviction și fallback;
- pool acquisition time și saturation;
- DB locks, deadlocks, slow queries, WAL și bloat;
- Elasticsearch indexing/search latency, rejects și unassigned shards;
- object upload success și processing age;
- restore success și RTO măsurat.

Alertarea trebuie să indice impact și acțiune, nu fiecare fluctuație.

22.2 Capacity model

Estimează:

- rows/documents/objects pe zi;
- bytes per item plus index/replica overhead;
- read/write QPS și burst;
- retention și growth;
- working set și cache;

- network pentru replicare și reindex;
- headroom pentru failover;
- cost de backup și restore;
- cost per query sau per tenant.

Validează estimarea prin load test și producție. Recalculează înainte ca retenția sau traficul să se dubleze.

22.3 Diagnostic sistematic

- 1 Confirmă impactul și intervalul.
- 2 Compară cu un baseline bun.
- 3 Urmărește request-ul între servicii.
- 4 Separă queueing, compute, network și storage.
- 5 Verifică saturarea și schimbările recente.
- 6 Mitighează sigur.
- 7 Păstrează dovezi pentru root cause.
- 8 Adaugă acțiuni care previn sau detectează repetarea.

Gate 22

Construiește un dashboard unic pentru cele trei stores principale. Injectează pool exhaustion, query lent, cache outage, CDC lag și disk pressure. Pentru fiecare, scrie alertă, triage și runbook.

Capitolul 23 - Testarea sistemelor de date

23.1 Piramida relevantă

- unit tests pentru mapping și logică;
- integration tests cu motoare reale, nu substituenți incompatibili;
- contract tests pentru evenimente și scheme;
- migration tests de la versiuni reale;
- property-based tests pentru invariants;
- concurrency tests;
- load/soak tests;
- chaos/failure injection;
- backup/restore tests;
- reconciliation tests.

H2 nu este PostgreSQL. Un mock Elasticsearch nu verifică analyzer, mapping, ranking sau compatibilitate. Folosește Testcontainers sau medii efemere pentru fidelity.

23.2 Test data

Ai nevoie de:

- dataset mic, controlat, pentru corectitudine;
- generator determinist pentru volum;
- skew, hot keys, nulls și valori-limită;
- Unicode și limbi reale;
- evenimente out-of-order și duplicate;
- documente mari și mappings neașteptate;
- date anonimizate, nu dump-uri de producție necontrolate.

23.3 Performance discipline

Specifică hardware, versiune, configurație, volum, warm-up, concurență și percentile. Nu compara tehnologii pe laptop cu setări diferite și apoi generaliza. Păstrează scripturile și rezultatele brute.

Gate 23

Fă o suită care pornește PostgreSQL, Redis și Elasticsearch reale, aplică migrații, indexează date, rulează relevance tests și simulează failure/retry. Rulează-o în CI pe un subset și periodic complet.

Capitolul 24 - System design și colaborarea cu business-ul

24.1 Întrebări obligatorii

Când auzi „avem nevoie de Elasticsearch” sau „baza nu scalează”, întreabă:

- ce comportament nu poate fi livrat acum?
- care sunt query-urile și distribuția lor?
- ce latență, prospețime și relevanță sunt acceptabile?
- care este volumul și creșterea?
- ce date sunt autoritative?
- ce se întâmplă când noul sistem este indisponibil?
- cine îl operează și care este bugetul?
- putem obține 80% din rezultat optimizând sistemul existent?

24.2 Documente profesionale

Pentru fiecare proiect, păstrează:

- problem statement și success metrics;
- ADR cu opțiuni și trade-offs;

- data model și data flow;
- schema/mapping/event contract;
- capacity estimate;
- threat model;
- test plan și benchmark;
- rollout/rollback;
- dashboards și runbooks;
- postmortem pentru defecte importante.

24.3 Cum comunică limitele

Înlocuiește „nu se poate” cu o frontieră măsurabilă: „Putem oferi rezultate în sub 300 ms și actualizare sub 10 secunde; consistența imediată ar necesita citire din PostgreSQL și ar reduce opțiunile de ranking”. Stakeholderul poate decide când vede costul și efectul.

Gate 24

Prezintă designul unui catalog către două audiențe: 10 minute pentru business și 30 de minute pentru engineering. Include alternative, metrice, riscuri, cost, plan incremental și criteriu de oprire.

Plan de studiu pe 40 de săptămâni

Săptămâni	Temă	Livrabil verificabil
1-2	Alegerea store-ului, model mental	3 ADR-uri și diagnostic
3-5	Model relațional și SQL	Schemă marketplace + 20 query-uri
6-7	PostgreSQL internals	Laborator MVCC/WAL/vacuum
8-10	Indexuri și plans	Raport 10 query-uri optimizate
11-12	Tranzacții și concurență	Rezervări fără oversell
13-14	Spring data access	API, pool, JPA/SQL, Testcontainers
15	Migrații și backfills	Expand-contract fără downtime
16	Backup, PITR, HA	Restore drill și runbook
17-18	Redis	Cache rezilient și benchmark
19-21	Elasticsearch fundamentals	Catalog indexat și benchmark shards
22-23	Mapping și analyzers	Mapping versionat + teste
24-26	Relevanță și Query DSL	Golden set și raport NDCG/MRR
27-28	CDC și outbox	DB-to-index convergent
29	Elasticsearch operations	ILM, snapshot, failure drill
30	Object storage	Upload securizat și lifecycle

Săptămâni	Temă	Livrabil verificabil
31	MongoDB	Model comparativ document/relational
32	Cassandra	Model time-bucket și compaction lab
33	Time-series/OLAP	Benchmark Timescale/ClickHouse
34	Vector/hybrid	Evaluare pgvector versus Elastic
35	Security și multi-tenancy	Threat model + leakage tests
36	Observabilitate și capacity	Dashboard, SLO și cost model
37	Reliability testing	Failure suite și restore automat
38-40	Capstone și interviu	Proiect, demo, postmortem, prezentare

Ritmul fiecărei faze

La începutul săptămânii definește o întrebare și o metrică. La final trebuie să existe un commit, un experiment reproductibil și o explicație scrisă. La fiecare patru săptămâni fă o demonstrație fără notițe și elimină o tehnologie/configurație care nu are justificare.

Proiectul 1 - Catalog de produse production-grade

Arhitectură

- PostgreSQL: produse, variante, preț, stoc și outbox;
- Spring Boot: API, validare, RBAC și idempotency;
- Debezium/Kafka: transportul schimbărilor;
- Elasticsearch: full-text, filtre, autocomplete și agregări;
- Redis: cache-aside pentru product detail și rate limiting;
- S3-compatible storage: imagini și documente;
- OpenTelemetry/metrics/logs: trasabilitate.

Cerințe

- multi-tenant fără leakage;
- create/update/delete converg în search;
- alias-based zero-downtime reindex;
- relevance golden set de minimum 100 queries;
- upload direct, checksum și scanare;
- p95 declarat la trafic declarat;
- rebuild complet și reconciliation;

- backup/restore pentru DB și snapshots pentru index;
- dashboard și patru runbooks.

Demo de incident

Oprește Elasticsearch 15 minute, continuă update-urile, apoi recuperează. Arată lag-ul, retry, idempotency și convergența. După aceea corupe mapping-ul într-un index de test și reconstruiește prin alias.

Proiectul 2 - Platformă multi-tenant de document search

Flux

- 1 API emite URL de upload limitat.
- 2 Clientul urcă documentul în object storage.
- 3 Worker validează checksum, tip, malware și status.
- 4 Parserul extrage text și metadata.
- 5 PostgreSQL păstrează ownership, ACL, workflow și audit.
- 6 Elasticsearch păstrează chunks, fields și filtre de securitate.
- 7 Opțional, embeddings permit hybrid search.

Cerințe

- autorizare înainte de emiterea URL-ului și înainte de download;
- ACL aplicat în orice search;
- parsare reluabilă și versionată;
- deduplicare content hash;
- reprocessing cu versiune nouă de parser/model;
- lexical baseline înainte de vector search;
- retention și delete propagate;
- test cu 100.000 documente și tenants dezechilibrați;
- threat model și audit.

Dovezi

Publică precision/recall/NDCG, p95, freshness, storage per document, cost estimat, failure matrix și un postmortem al unei erori injectate.

Proiectul 3 - Analytics operațional din CDC

Arhitectură

- PostgreSQL este source of truth pentru comenzi;

- Debezium captează schimbările;
- Kafka păstrează fluxul și permite replay;
- ClickHouse sau TimescaleDB servește dashboard-urile;
- job-ul de reconciliation compară totals cu PostgreSQL;
- BI/dashboard citește numai store-ul analitic.

Cerințe

- snapshot inițial plus streaming fără gap;
- schema evolution compatibilă;
- evenimente duplicate și out-of-order;
- latență de prospecție măsurată;
- backfill pentru o coloană derivată;
- late-arriving updates;
- retenție și compresie;
- load test și capacity estimate;
- restore/rebuild din sursa autoritativă.

Checklist de competență

SQL și PostgreSQL

- [] Modelez chei, relații și invariants.
- [] Scriu joins, windows, CTE și keyset pagination.
- [] Citesc **EXPLAIN ANALYZE BUFFERS**.
- [] Aleg între B-tree, GIN, GiST și BRIN.
- [] Explic MVCC, WAL, vacuum și bloat.
- [] Proiectez locking și idempotency.
- [] Fac migrații expand-contract și backfill reluabil.
- [] Execut PITR și măsoar RPO/RTO.

Redis și Elasticsearch

- [] Proiectez cache-aside, TTL, jitter și fallback.
- [] Explic eviction, persistence, Sentinel și Cluster.
- [] Modelez text/keyword/nested și analyzers.
- [] Scriu bool queries, filters, aggregations și search-after.
- [] Evaluez BM25/relevanța cu un golden set.
- [] Aleg shard/replica/ILM prin benchmark.

- [] Fac reindex prin alias și snapshot restore.
- [] Sincronizez prin outbox/CDC și reconciliesc.

Data platform

- [] Aleg embed/reference și o shard key MongoDB.
- [] Modelez partition/clustering keys Cassandra.
- [] Construiesc upload multipart securizat.
- [] Explic snapshot/offset/schema evolution Debezium.
- [] Separ OLTP de time-series și OLAP.
- [] Evaluatez exact/ANN și hybrid search.
- [] Aplic tenant isolation, retention și delete end-to-end.
- [] Construiesc SLO, dashboards și runbooks.

Întrebări de interviu și răspunsuri așteptate

Design și SQL

- 1 **Când preferi PostgreSQL în loc de MongoDB?** Discuți relații, invariants, query-uri, tranzații, flexibilitate, scală și operare.
- 2 **De ce nu este folosit indexul?** Selectivitate mică, statistică, cast/funcție, ordine compusă, cost random I/O, query care citește mare parte din tabel.
- 3 **Cum previi oversell?** Update condiționat/lock/version, tranzație scurtă, retry și idempotency.
- 4 **De ce replicarea nu este backup?** Propagă erori și nu oferă neapărat istoric izolat; ai nevoie de retenție și restore testat.
- 5 **Cum migrezi o coloană fără downtime?** Expand-contract, cod compatibil, backfill reluabil, validate, switch, cleanup.

Redis și search

- 1 **Cum tratezi cache stampede?** TTL jitter, request coalescing/lock limitat, stale-while-revalidate, warming și protejarea originului.
- 2 **text versus keyword?** Analiză/full-text versus valoare exactă pentru filter/sort/aggregation.
- 3 **Cum alegi numărul de shards?** Benchmark cu volum, ingestie, query fan-out, noduri, recovery și retenție; fără constantă magică.
- 4 **Cum măsoară search quality?** Golden judgments, precision/recall/MRR/NDCG, segmente, zero-result și online metrics.
- 5 **Cum schimbi un mapping?** Index versionat, reindex, validate, alias atomic, cleanup ulterior.

Distribuție și reliability

- 1 **Cum îți PostgreSQL și Elasticsearch sincronizate?** Outbox/CDC, event version, consumer idempotent, retry/DLT, reconciliation și rebuild.

- 2 **Ce înseamnă eventual consistency pentru produs?** Staleness budget, UI behavior, freshness SLI și fallback.
- 3 **Cum alegi shard key Cassandra/MongoDB?** Query routing, cardinalitate, distribuție, hotspot, creștere și resharding.
- 4 **Ce testezi într-un restore drill?** Integritate, versiune, secrets, timp, replay delta, reconectare și documentarea RPO/RTO.
- 5 **Când alegi vector search?** Numai după lexical baseline și evaluare; discuți semantic need, cost, filtering, recall/latency și model versioning.

Greșeli frecvente

- polyglot persistence înainte de a exista un workload măsurat;
- Elasticsearch folosit drept source of truth;
- Redis fără **maxmemory**, TTL sau comportament de fallback;
- index pentru fiecare coloană și ignorarea costului de write;
- **SELECT ***, offset pagination și N+1 în căi fierbinți;
- DDL mare executat în același moment cu deploy-ul incompatibil;
- dual write către DB și index fără outbox/reconciliation;
- relevanță reglată după impresii, fără golden set;
- număr de shards copiat dintr-un blog;
- backup bifat, restore neexecutat;
- logs cu date sensibile;
- shared schema fără tenant filter garantat;
- CDC slot neobservat care umple discul;
- Cassandra aleasă înaintea query-urilor;
- vector search adăugat pentru marketing, fără benchmark;
- load test cu date uniforme când producția are skew și hot keys.

Resurse și bibliografie oficială

PostgreSQL și MySQL

- [S1] PostgreSQL, MVCC Introduction: <https://www.postgresql.org/docs/current/mvcc-intro.html>
- [S2] PostgreSQL, Routine Vacuuming: <https://www.postgresql.org/docs/current/routine-vacuuming.html>
- [S3] PostgreSQL, Using EXPLAIN: <https://www.postgresql.org/docs/current/using-explain.html>
- [S4] PostgreSQL, Index Types: <https://www.postgresql.org/docs/current/indexes-types.html>

- [S5] PostgreSQL, Transaction Isolation: <https://www.postgresql.org/docs/current/transaction-iso.html>
- [S6] PostgreSQL, Explicit Locking: <https://www.postgresql.org/docs/current/explicit-locking.html>
- [S7] PostgreSQL, Continuous Archiving and PITR: <https://www.postgresql.org/docs/current/continuous-archiving.html>
- [S8] PostgreSQL, SQL VACUUM: <https://www.postgresql.org/docs/current/sql-vacuum.html>
- [S9] MySQL 8.4 Reference Manual: <https://dev.mysql.com/doc/refman/8.4/en/>
- [S10] MySQL, InnoDB and Replication: <https://dev.mysql.com/doc/refman/8.4/en/innodb-and-mysql-replication.html>
- [S11] MySQL, Backup and Recovery: <https://dev.mysql.com/doc/refman/8.4/en/backup-and-recovery.html>

Redis

- [S12] Redis Sentinel: https://redis.io/docs/latest/operate/oss_and_stack/management/sentinel/
- [S13] Redis Replication: https://redis.io/docs/latest/operate/oss_and_stack/management/replication/
- [S14] Redis Data Types: <https://redis.io/docs/latest/develop/data-types/compare-data-types/>
- [S15] Redis Key Eviction: <https://redis.io/docs/latest/develop/reference/eviction/>
- [S16] Redis Cache-Aside: <https://redis.io/docs/latest/develop/use-cases/cache-aside/>
- [S17] Redis Persistence: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- [S18] Spring Data Redis Reference: <https://docs.spring.io/spring-data/redis/reference/redis.html>
- [S19] Redis TLS: https://redis.io/docs/latest/operate/oss_and_stack/management/security/encryption/

Elasticsearch

- [S20] Elastic, Analyzer: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/analyzer>
- [S21] Elastic, Dynamic Mapping: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/dynamic>
- [S22] Elastic, Keyword Type: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/keyword>
- [S23] Elastic, Multi-fields: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/multi-fields>
- [S24] Elastic, Boolean Query: <https://www.elastic.co/docs/reference/query-languages/query-dsl/query-dsl-bool-query>

- [S25] Elastic, Full-text Queries: <https://www.elastic.co/docs/reference/query-languages/query-dsl/full-text-queries>
- [S26] Elastic, Similarity / BM25: <https://www.elastic.co/docs/reference/elasticsearch/index-settings/similarity>
- [S27] Elastic, Index Lifecycle Management: <https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management>
- [S28] Elastic, ILM Phases and Actions: <https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management/index-lifecycle>
- [S29] Elastic, Create Snapshots: <https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore/create-snapshots>
- [S30] Elastic, Size Your Shards: <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/size-shards>
- [S31] Spring Data Elasticsearch, Repositories: <https://docs.spring.io/spring-data/elasticsearch/reference/elasticsearch/repositories/elasticsearch-repositories.html>
- [S32] Spring Data Elasticsearch, Operations: <https://docs.spring.io/spring-data/elasticsearch/reference/elasticsearch/template.html>

NoSQL, CDC și vectori

- [S33] MongoDB, Embedded Data Models: <https://www.mongodb.com/docs/manual/data-modeling/embedding/>
- [S34] MongoDB, Transactions: <https://www.mongodb.com/docs/manual/data-modeling/enforce-consistency/transactions/>
- [S35] MongoDB, Sharding: <https://www.mongodb.com/docs/manual/sharding/index.html>
- [S36] Debezium, Transformations: <https://debezium.io/documentation/reference/stable/transformations/index.html>
- [S37] Debezium, Outbox Event Router: <https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html>
- [S38] pgvector Official Repository: <https://github.com/pgvector/pgvector>
- [S39] Apache Cassandra, Architecture Overview: <https://cassandra.apache.org/doc/stable/cassandra/architecture/overview.html>
- [S40] Apache Cassandra, Data Definition: <https://cassandra.apache.org/doc/latest/cassandra/developing/cql/ddl.html>
- [S41] Apache Cassandra, Compaction Overview: <https://cassandra.apache.org/doc/stable/cassandra/managing/operating/compaction/overview.html>

Object storage, time-series și hybrid search

- [S42] Amazon S3, Presigned URLs: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>
- [S43] Amazon S3, Multipart Upload: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html>

- [S44] Amazon S3, Replication: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html>
- [S45] Amazon S3, Object Lock: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>
- [S46] Amazon S3, Security Best Practices: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/security-best-practices.html>
- [S47] Timescale, Hypertables: <https://docs.timescale.com/use-timescale/latest/hypertables/>
- [S48] Timescale, Continuous Aggregates: <https://docs.timescale.com/use-timescale/latest/continuous-aggregates/about-continuous-aggregates/>
- [S49] Elastic, kNN Query: <https://www.elastic.co/docs/reference/query-languages/query-dsl/query-dsl-knn-query>
- [S50] Elastic, Hybrid Search: <https://www.elastic.co/docs/solutions/search/hybrid-search>

Ordinea recomandată a certificării personale

Nu ai nevoie de o certificare comercială ca să începi. Folosește această ordine a dovezilor:

- 1 SQL și PostgreSQL demonstrate prin planuri și tranzacții.
- 2 Restore drill și migrare fără downtime.
- 3 Redis cu failure behavior măsurat.
- 4 Elasticsearch cu relevance evaluation și operare.
- 5 Outbox/CDC cu convergență și rebuild.
- 6 Object storage și securitate multi-tenant.
- 7 Un singur store specializat suplimentar, ales pentru un workload real.
- 8 Capstone prezentat ca sistem de produs, nu ca listă de tehnologii.

Un portofoliu puternic conține rezultate reproductibile: cod, set de date, benchmark, dashboard, ADR, failure demo și runbook. Aceasta este diferența dintre „am folosit Elasticsearch” și „pot proiecta și opera o platformă de date”.